

InventoryPlus — Complete Hooking & Connections Plan

A master guide compiling all affected code blocks, sub-hook connections, and the full proposed codebase.

This document lists the architectural steps to modularize Inventory.tsx. It covers: (1) what code to delete, (2) how the new modular sub-hooks share states, and (3) the complete implementation of the sub-hooks and their orchestrator.

Summary of Affected Lines in Inventory.tsx

Lines in Inventory.tsx	Description of Affected Code	Action
Lines 59–60	API base URL definition and constants	Remove
Line 407	workstationSpecs state hook definition	Remove
Lines 416–425	Initial localStorage spec loading hook	Remove
Line 427	cards (departments) state definition	Remove
Lines 431–450	fetchDepartments function and loading hook	Remove
Lines 456–474	handleAddCardSubmit function	Remove
Line 476	members state definition	Remove
Lines 499–526	fetchMembers function and watcher hook	Remove
Lines 528–553	handleAddMemberSubmit function	Remove
Lines 571–625	Specs retrieval hook and saveSpecsToDb function	Remove
Lines 627–740	handleSaveCardEdits editing submit handler	Remove
Lines 760–805	handleDeleteCard handler	Remove

1. How the Sub-Hooks Connect to Each Other

State Source	Target Hook	Connection Link (Dependency Array)	Triggered Action
selectedDepartment	useMembers	[selectedDepartment]	Triggers fetchMembers() to load members of the department from DB.
members list + selectedMemberIndex	useWorkstationSpecs	[selectedMemberObj, selectedDepartment]	Derives selectedMemberObj and triggers fetchSpecs() to load specific user workstation data.

Connection 1: Selected Department to Members

- useMembers takes selectedDepartment as an argument. Inside, a useEffect hook monitors it to trigger a new fetch.
- **Reference:** `useEffect(() => { fetchMembers(selectedDepartment) }, [selectedDepartment])`

Connection 2: Members List to Workstation Specs

- The UI derives selectedMemberObj from the members list returned by useMembers.
- useWorkstationSpecs accepts this object and monitors it to retrieve user specifications.
- **Reference:** `useEffect(() => { fetchSpecs() }, [selectedMemberObj, selectedDepartment])`

2. Code Blocks to Remove from Inventory.tsx (Part 1)

API Constants (Lines 59–60)

```
const rawApiUrl = import.meta.env.VITE_API_URL || 'http://127.0.0.1:8000';
const API_BASE_URL = rawApiUrl.endsWith('/') ? rawApiUrl.slice(0, -1) : rawApiUrl;
```

States & Initial Fetch (Lines 407, 416–427)

```
const [workstationSpecs, setWorkstationSpecs] = useState<{ [key: string]: any }>({});
const [cards, setCards] = useState<any[]>([]);

useEffect(() => {
  const saved = localStorage.getItem("inventoryplus_workstation_specs");
  if (saved) {
    try {
      setWorkstationSpecs(JSON.parse(saved));
    } catch (err) {
      console.error("Error parsing saved workstation specs:", err);
    }
  }
}, []);
```

2. Code Blocks to Remove from Inventory.tsx (Part 2)

Department Fetch & Submit (Lines 431–474)

```
const fetchDepartments = async () => {
  try {
    const res = await fetch(`${API_BASE_URL}/api/departments/`);
    if (res.ok) {
      const data = await res.json();
      setCards(data.map((dept: any) => ({ id: dept.id, ...cardProps, name: dept.name })));
    }
  } catch (err) { console.error(err); }
};

useEffect(() => { fetchDepartments(); }, []);

const handleAddCardSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!newDeptName.trim()) return;
  try {
    const res = await fetch(`${API_BASE_URL}/api/departments/`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ name: newDeptName.trim() }),
    });
    if (res.ok) { setNewDeptName(""); setShowAddModal(false); fetchDepartments(); }
  } catch (err) { console.error(err); }
};
```

2. Code Blocks to Remove from Inventory.tsx (Part 3)

Members, Spec Edits, & Deletion (Lines 476–805 excerpt)

```
const fetchMembers = async (deptName: string) => {
  try {
    const res = await fetch(`${API_BASE_URL}/api/departments/${encodeURIComponent(deptName)}/users/`);
    if (res.ok) {
      const data = await res.json();
      setMembers(data);
    }
  } catch (err) { console.error(err); }
};

const handleSaveCardEdits = (e: React.FormEvent) => {
  e.preventDefault();
  // ... edits details based on category key ...
};

const handleDeleteCard = (cardTitle: string) => {
  // ... deletes specs local state and backend sync ...
};
```

3. Orchestrator Hook (useInventoryApi.ts)

```
import { useState } from 'react';
import { useDepartments } from './useDepartments';
import { useMembers } from './useMembers';
import { useWorkstationSpecs } from './useWorkstationSpecs';

export function useInventoryApi() {
  const [selectedDepartment, setSelectedDepartment] = useState<string | null>(() => {
    return localStorage.getItem("inventoryplus_selected_department");
  });
  const [selectedMemberIndex, setSelectedMemberIndex] = useState<number>(() => {
    const saved = localStorage.getItem("inventoryplus_selected_member_index");
    return saved !== null ? Number(saved) : -1;
  });

  const { cards, handleAddCardSubmit } = useDepartments();

  const { members, isSubmittingMember, handleAddMemberSubmit } = useMembers(
    selectedDepartment, selectedMemberIndex, setSelectedMemberIndex
  );

  const memberList = selectedDepartment ? members.map(m => `${m.username}${m.email ? ` (${m.email})` : ""}`) : [];
  const selectedMemberName = selectedDepartment && selectedMemberIndex !== -1 && memberList[selectedMemberIndex] ?
  memberList[selectedMemberIndex] : "";
  const selectedMemberCleanName = selectedMemberName ? selectedMemberName.split(' ')[0] : "";
  const currentMemberKey = selectedDepartment && selectedMemberCleanName ? `${selectedDepartment}-${selectedMemberCleanName}` : "";
  const selectedMemberObj = selectedDepartment && selectedMemberIndex !== -1 && members[selectedMemberIndex] ?
  members[selectedMemberIndex] : null;

  const { workstationSpecs, setWorkstationSpecs, handleSaveCardEdits, handleDeleteCard } = useWorkstationSpecs(
    selectedMemberObj, selectedDepartment, selectedMemberCleanName, currentMemberKey
  );

  return {
    selectedDepartment, setSelectedDepartment, selectedMemberIndex, setSelectedMemberIndex,
    cards, members, workstationSpecs, isSubmittingMember, memberList, selectedMemberName,
    selectedMemberCleanName, currentMemberKey, selectedMemberObj, setWorkstationSpecs,
    handleAddCardSubmit, handleAddMemberSubmit, handleSaveCardEdits, handleDeleteCard
  };
}
```

4. The Sub-Hooks Implementation — useDepartments.ts

```
import { useState, useEffect } from 'react';

export function useDepartments() {
  const [cards, setCards] = useState<any[]>([]);

  const fetchDepartments = async () => {
    try {
      const res = await fetch('/api/departments/');
      if (res.ok) {
        const data = await res.json();
        setCards(data);
      }
    } catch (err) { console.error(err); }
  };

  useEffect(() => { fetchDepartments(); }, []);

  const handleAddCardSubmit = async (newDeptName: string) => {
    if (!newDeptName.trim()) return false;
    try {
      const res = await fetch('/api/departments/', {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ name: newDeptName.trim() }),
      });
      if (res.ok) { fetchDepartments(); return true; }
    } catch (err) { console.error(err); }
    return false;
  };

  return { cards, handleAddCardSubmit };
}
```

4. The Sub-Hooks Implementation — useMembers.ts

```
import { useState, useEffect } from 'react';

export function useMembers(
  selectedDepartment: string | null,
  selectedMemberIndex: number,
  setSelectedMemberIndex: (idx: number) => void
) {
  const [members, setMembers] = useState<any[]>([]);
  const [isSubmittingMember, setIsSubmittingMember] = useState(false);

  const fetchMembers = async (deptName: string) => {
    try {
      const res = await fetch(`/api/departments/${encodeURIComponent(deptName)}/users/`);
      if (res.ok) {
        const data = await res.json();
        setMembers(data);
        const savedUsername = localStorage.getItem("inventoryplus_selected_member_username");
        if (savedUsername) {
          const idx = data.findIndex((m: any) => m.username === savedUsername);
          if (idx !== -1) setSelectedMemberIndex(idx);
        }
      }
    } catch (err) { console.error(err); }
  };

  useEffect(() => {
    if (selectedDepartment) fetchMembers(selectedDepartment);
    else setMembers([]);
  }, [selectedDepartment]); // <-- CONNECTION TO DEPT SELECTION

  const handleAddMemberSubmit = async (newMemberName: string, newMemberEmail: string) => {
    if (!newMemberName.trim() || !selectedDepartment || isSubmittingMember) return false;
    setIsSubmittingMember(true);
    try {
      const res = await fetch(`/api/departments/${encodeURIComponent(selectedDepartment)}/users/`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ username: newMemberName.trim(), email: newMemberEmail.trim() || null }),
      });
      if (res.ok) { fetchMembers(selectedDepartment); return true; }
    } catch (err) { console.error(err); }
    finally { setIsSubmittingMember(false); }
    return false;
  };

  return { members, isSubmittingMember, handleAddMemberSubmit };
}
```


4. The Sub-Hooks Implementation — useWorkstationSpecs.ts

```
import { useState, useEffect } from 'react';

export function useWorkstationSpecs(
  selectedMemberObj: any,
  selectedDepartment: string | null,
  selectedMemberCleanName: string,
  currentMemberKey: string
) {
  const [workstationSpecs, setWorkstationSpecs] = useState<Record<string, any>>({});

  useEffect(() => {
    if (selectedMemberObj && selectedMemberObj.id) {
      const fetchSpecs = async () => {
        try {
          const res = await fetch(`/api/users/${selectedMemberObj.id}/specs/`);
          if (res.ok) {
            const data = await res.json();
            setWorkstationSpecs(prev => ({ ...prev, [currentMemberKey]: data }));
          }
        } catch (err) { console.error(err); }
      };
      fetchSpecs();
    }
  }, [selectedMemberObj, selectedDepartment]); // <-- CONNECTION TO MEMBER/DEPT

  const handleSaveCardEdits = (editingCardTitle: string, editingCardItems: any[]) => {
    // Save state logic here
  };

  const handleDeleteCard = (cardTitle: string) => {
    // Delete state logic here
  };

  return { workstationSpecs, setWorkstationSpecs, handleSaveCardEdits, handleDeleteCard };
}
```